XMODEM C++ Design Document

Checksum-based implementation for TRS-80 interoperability

Sharky J Yellowfish
Screaming Yellow Fish Engineering

March 9, 2026

On-the-wire framing (checksum XMODEM, 128-byte blocks)

Each data block is:

SOH 0x01
BLK 1 byte (block number, wraps 0xFF→0x00)
~BLK 1 byte (bitwise complement of BLK)
DATA 128 bytes
CHECKSUM 1 byte = (sum(DATA) mod 256)

Control bytes:

- SOH=0x01, EOT=0x04, ACK=0x06, NAK=0x15, CAN=0x18

Timeouts / retry policy

- **Wait for block start (SOH/EOT):** ~30 seconds, ignore 0x00 (NUL) bytes.
- **Byte-level reads inside a block (blk#, complement, data, checksum):** ~3 seconds per byte.
- **On receive errors:** send NAK, retry up to **10** times, then abort.
- **On send:** after sending a block, wait for ACK (~30 seconds). On timeout/other response, retry up to **10** times, then abort.
- **Strict block numbering on receive:**
    - If received BLK == last_good: treat as **duplicate** (sender didn't see ACK) → **ACK again**, do not write.
    - If received BLK == last_good + 1: accept/write, **ACK**.
    - Else: **CANCEL** (send CAN) and abort (no attempt to resync).

Padding behavior

When sending the final partial block, pad the remaining bytes of the 128-byte block with **0x00**.

Receiver algorithm

State variables:

- last_good_blk (byte), initial 0
- expected_blk = last_good_blk + 1 (byte wrap)
- retry = 0..10

1. **Initialize / start transfer**
    - Open output file for write.
    - Flush/discard any pending serial input.
    - Send initial NAK.
2. **Main loop**
    - Read next non-NUL byte with **30s timeout**:
        - If SOH:
            1. Read blk (3s timeout)
            2. Read blk_cpl (3s timeout), verify blk_cpl == (~blk & 0xFF) else error
            3. Read 128 data bytes (3s timeout each), compute checksum
            4. Read rx_checksum (3s timeout), compare
            5. If checksum ok:
                - If blk == last_good_blk: **ACK**, reset retry, continue
                - Else if blk == last_good_blk + 1: write 128 bytes, set last_good_blk=blk, **ACK**, reset retry, continue
                - Else: send **CAN** and abort
            6. If any failure: do “line clear” (see below), send NAK, retry++, abort if retry>=10
        - If EOT:
            - Flush/close output file
            - Send ACK

        - Success return
    - Else:
        - Treat as error: “line clear”, send NAK, retry++, abort if retry>=10

“Line clear” behavior: before sending NAK/CAN, the code reads/discards incoming bytes until it sees about **3 seconds of silence**, then transmits the control byte. This reduces collisions when the line is noisy or the other side is still dumping bytes.

Sender algorithm

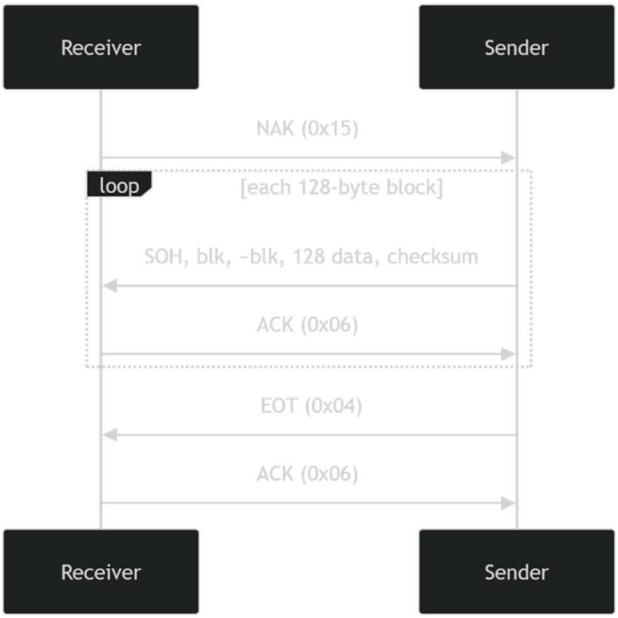
State variables:

- blk = 1 (byte wrap)
- per-block retry = 0..10

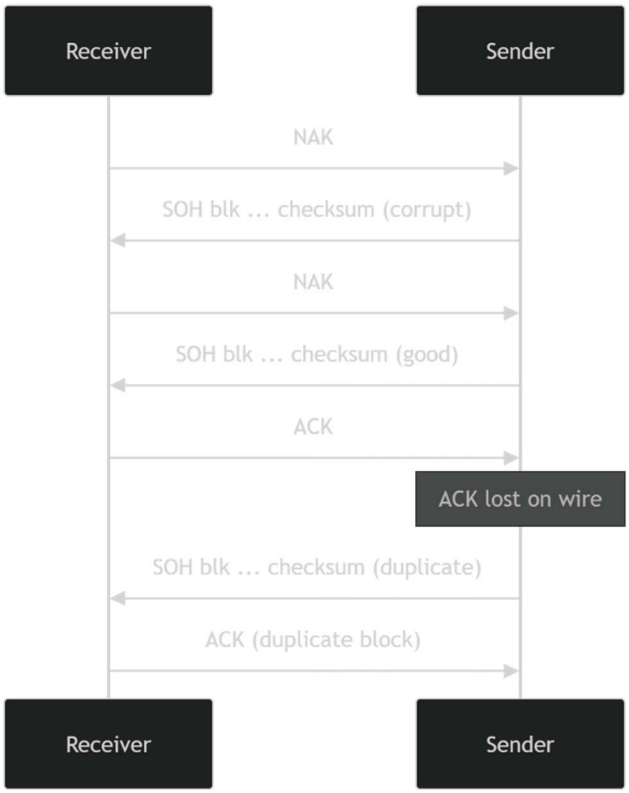
1. **Handshake**
    - Open input file.
    - Wait for NAK from receiver (the ASM loops with a short timeout repeatedly). On CAN, abort.
2. **For each 128-byte chunk**
    - Read up to 128 bytes; if EOF → go to EOT step.
    - If short read, pad remainder with 0x00.
    - Build frame: SOH, blk, ~blk, data[128], checksum.
    - Send frame.
    - Wait up to ~30s for response:
        - ACK: blk++, proceed to next block
        - NAK or timeout/garbage: resend same block (retry++, abort at 10)
        - CAN: abort
3. **Finish**
    - Send EOT
    - Wait for ACK, retry sending EOT if necessary (up to ~10), else abort.

Sequence diagrams

Normal transfer

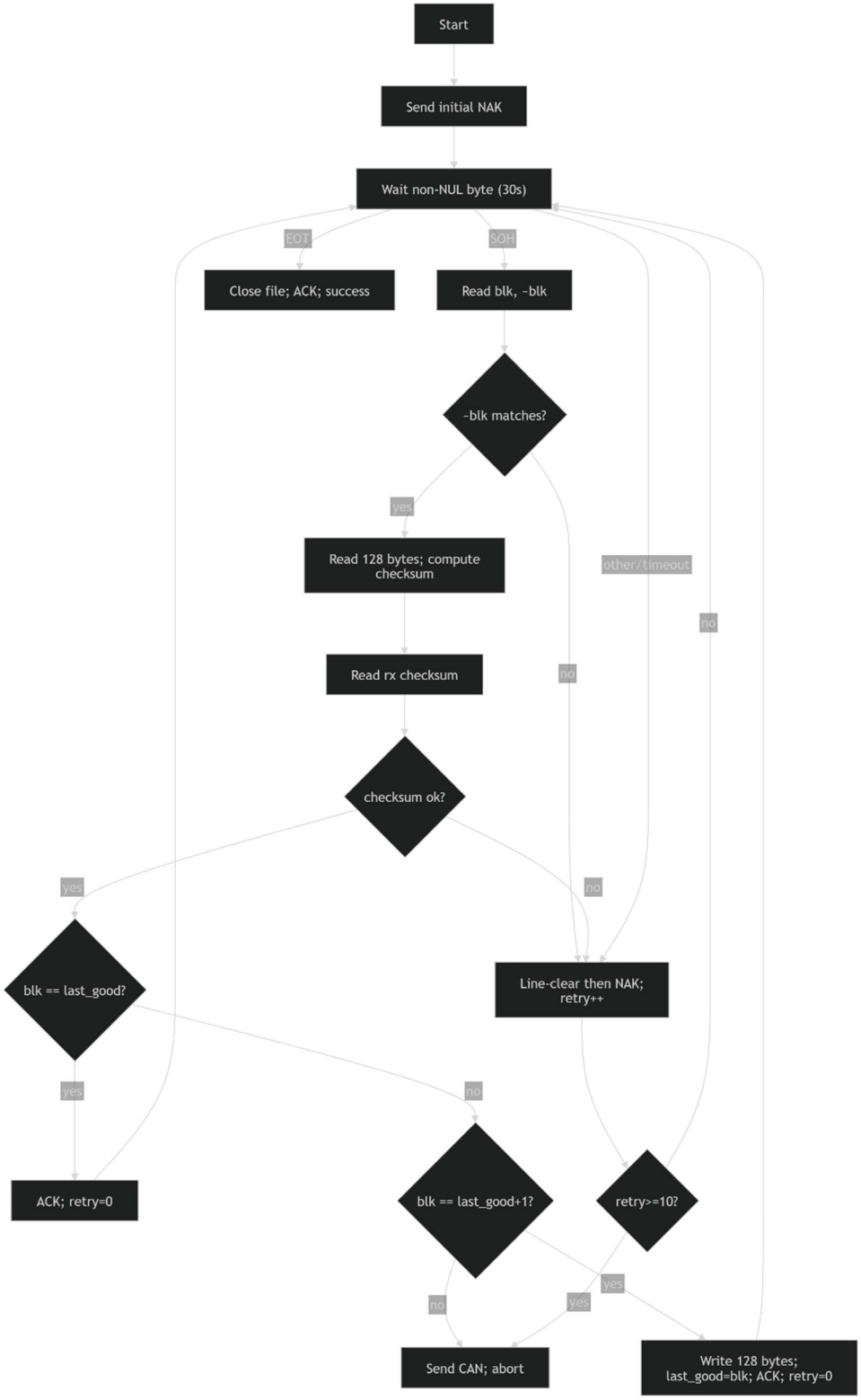


Corrupt block → NAK → retransmit → duplicate ACK case



Flowcharts

Receiver



Sender

Start
Wait for NAK
NAK
blk=1
Read up to 128 bytes
<128
Pad with 0x00
128
CAN
EOF
Send SOH blk ~blk data checksum
Send EOT; wait ACK
Wait resp (30s)
retry<10
ACK
timeout/NAK
retry<10
ACK
NAK/timeout/other
Success
retry++ ; resend EOT
blk++ ; next block
retry++ ; resend same blk
retry>=10
retry>=10
Abort